

Stylizing 3D Animations

DE CASTRO LEAL HENRIQUES Francisco

Alice JEANNIN

Mathieu SENART

June 28, 2026

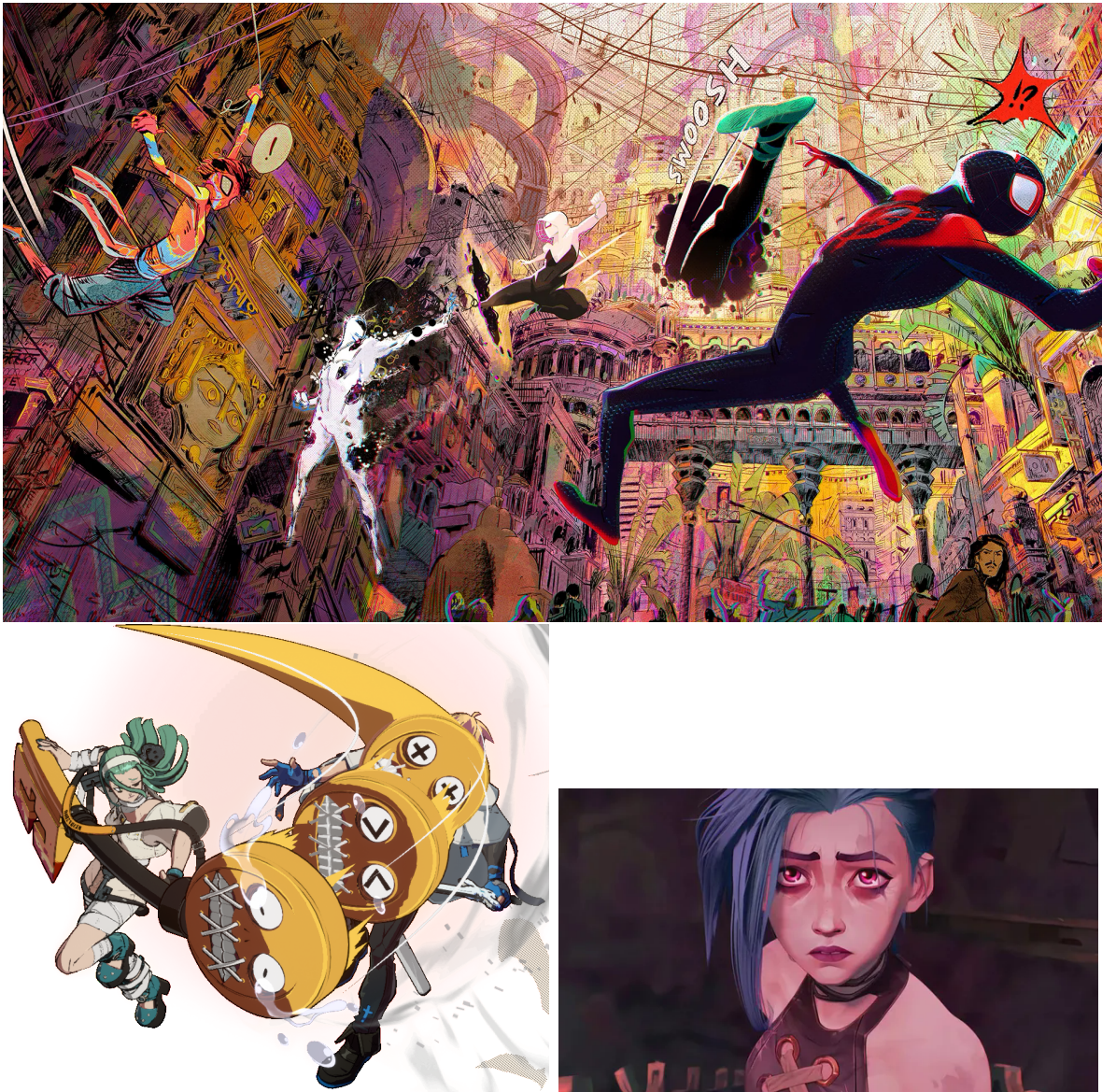


Figure 1: Examples of stylized animation across different mediums: Movies (Spider-Man: Across the Spider-Verse), Games (Guilty Gear: Strive), and Series (Arcane).

1 Introduction

Traditional 3D rendering has evolved throughout the years towards photorealism, in an attempt to create virtual scenes as close as possible to real life. However, there has always been the interest in the field of Computer Graphics to generate more artistic scenes as well: adding expressive stylization elements that drift away from reality to convey a certain artistic view.

However, many stylization techniques done today in the movie industry rely on hand-crafted post-processing effects, which can be very cumbersome and time-consuming. Our project, then, seeks to explore and implement solutions that apply stylization to the final render in an more automatic way — while also allowing the artist to adjust the final result — in earlier steps of the rendering pipeline.

In the end, we implemented a Graphics Renderer in C++, using OpenGL as our graphics API, Assimp to import pre-made animations to our program, and ImGui to display relevant debug information on the screen, such as FPS, the shaders that were loaded, etc. As our final result, we showcase a scene with some animations and stylizations applied to them.

2 Techniques implemented

2.1 Smear Frame Generation

We relied on [1] to implement the automatic generation of smear frames. In the paper, the authors provide a way to calculate motion offsets for each vertex in each frame of the animation, along with some stylization methods (elongated in-betweens, multiple in-betweens and motion lines), and a Python implementation of a Blender plugin. For our project, we imple-

mented the calculation of motion offsets, and stylization by elongated in-betweens.

To calculate motion offsets, we first select all the transformations for each bone joint throughout the full animation. Since the list of transformations for each articulation is defined through keyframes (which frequently do not match in time between joints), we calculate interpolated transforms at regular intervals of t ticks. The number of ticks per second is defined by the rendering engine.

After this step, we have a set of transformations for the full articulated character for each frame. We then select pairs of contiguous frames to calculate the motion offsets δ_{ik} , where i is a vertex and k is a frame. The calculation of motion offsets is done per-bone and consists, as a first step, of defining a motion ribbon [3] locally perpendicular to the motion vectors of the bone. We then assign a 1D parameter to each vertex with position p_i based on the bone axis:

$$u_{ik} = S\left(\frac{(p_i - c_{k,r}) \cdot \hat{b}}{\|b\|}\right)$$

where $c_{k,r}$ and $c_{k,t}$ are the positions of the root and the tip of the bone, respectively, $b = c_{k,t} - c_{k,r}$, $\hat{b} = \frac{b}{\|b\|}$, and S is a smoothstep function.

Next, we define a local direction of motion ($\hat{v}(u_{ik})$) by interpolating the motion direction of the root and the tip of the bone ($\hat{v}_{r,k}$ and $\hat{v}_{t,k}$), and use that to define the ribbon normal $\hat{n}_k$ for a parametric position u_{ik} along the bone axis. The motion offset is then given by:

$$\delta_{ik} = (p_i - c_{k,r}) \cdot \hat{n}_k(u_{ik})$$

Finally, we normalize δ_{ik} using a normalization term M_{ik} that considers all the vertices that are affected by bones connected either to the root or the tip of the current bone. This ensures that the stylization effect is applied smoothly across bones.

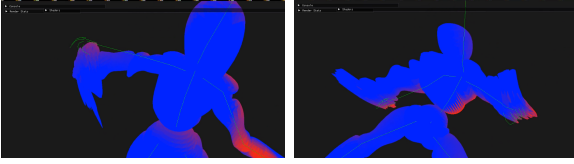


Figure 2: Examples of smear frames

2.1.1 Problems faced

To calculate the motion offsets, it was necessary to understand how the **Assimp** animation tree structure works. At first, we were not aware that *not* all nodes corresponded to joints, and we ended up using "joints" that didn't actually exist in the calculations, thus creating artifacts in the animation.

Another problem we encountered during development was figuring out exactly when an operation should be performed in world space or bone space. This caused issues for us for some time, as there were many operations being done in the wrong spaces, and we spent a considerable amount of time trying to extract useful information from logs and creating debug shaders to pinpoint where the calculations were failing.

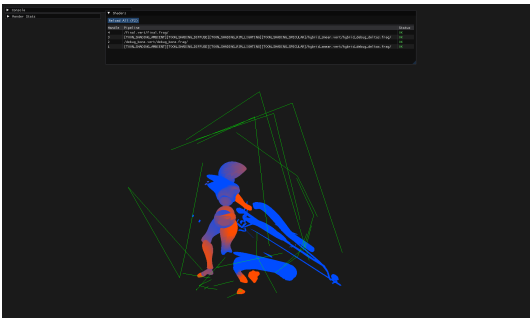


Figure 3: Bones showing (in green) in a debug shader after being calculated in the wrong space.

2.2 Silhouette stylization

2.2.1 Previous work

One of the stylization we wanted to implement is silhouette stylization. Silhouette drawing for non-photorealistic rendering (NPR) has been a topic for many research papers. Several different techniques have been found, with their advantages and inconvenients. The most popular technique, which is used in the classic toon shading, is to render the scene normally, then to translate and enlarge the mesh and render it in black with back-face culling inverted. This method has been shown by Raskar and Cohen [5], and Gooch et al. [gooch1999]. It is easy to implement and gives you nice results but you don't have much freedom to stylize it. Another possibility is to detect silhouette edges [markosian], ie edges which are part of a front-face and of a back-face triangle, and then to render the resulting polyline composed of the edges placed end to end. It is then easy to stylize the polyline with random offsets applied to its points, or with 2D brush texture. The last technique is to use image-base contour detection, by finding discontinuities in depth or normal buffer ([6], [4]). Polylines can then be extract with a sequential algorithm ([active-strokes]) using what Pierre B  nard et al. call "snakes".

2.2.2 Implemented work

As we wanted the rendering to be real time, we looked for GPU-compatible techniques, which excludes all kind of sequential algorithms. We implemented an image-base contour detection ([6], [4]) and then aimed to stylize it to have a sketchy effect, as the one of Curtis in its short films "Brick-a-brack" and "The new Chair" [2]. Curtis uses "particles of ink" subjected to a force field computed from the depth buffer to have this particular effect. For the sake of simplicity and performance, we imple-

mented something completely different using perlin noise, as explained in a later dedicated section.

2.2.3 Contour detection

We computed contour detection in a post-processing render pass. We used 3 different attributes to compute them in 3 different manners: depth, normals and the meshes' id, that is to say for each fragment the id of the mesh whose it is from (and 0 for the background which we initialized using `glClearBuffer`). In order to have all these attributes available in the our second render pass, we used multiple render targets (MPR) which consists of writing in several out textures in a single framebuffer (and thus in a single render pass). We then bound our 3 textures to the final render pass (see figure 4). The first one in the classical RGBA format for the rendering of the scene (which would be used or not in the final render pass depending on the shader choosen by the user). The second one contains unsigned int to save the mesh id. The third one is composed of vec4 of floats to save normals (rgb component) and depth (a component). We had to use a special format when creating the texture with `glTexImage2D` (`GL_RGBA32UI` and `GL_FLOAT`) in order not to have the value clamped to $[0,1]$ for the normals' components. It is worth to say that we didn't use directly the z component of the fragment position after perspective division because the precision was not enough for values far from the camera. Instead we passed the z component in the vertex shader as an out variable to interpolate it for each fragment without having the division by the w-component.

In the post-processing render pass, we compute the contours using the `texCoord` depending on the method:

- mesh id : contour if `meshId==0` and it exists neighbor s.t. `meshId!=0`

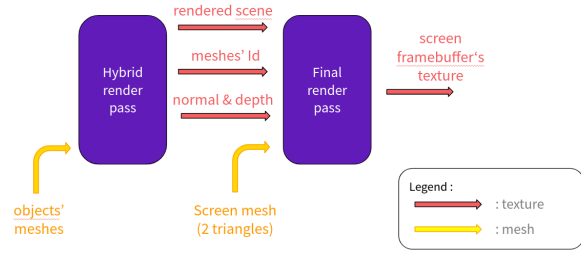


Figure 4: Our rendering pipeline

- depth : contour if exists neighbor s.t. $n_depth - depth > threshold$
- normal : contour if exists neighbor s.t. $dot(n_normal, normal) < threshold$ and $n_normal.z > normal.z$

The neighbors are found using `texelFetch` which takes integer couples as `texCoords` instead of float in $[0,1]$ like the traditional `texture` function. You can notice that we used extra condition (like `meshId==0` or the signed difference for depth) to have a 1-pixel wide contour thanks to non-symetric relations.

These 3 methods give different results and are complementary (see figure 5). For the sketchy line, we only used the depth method for artistic purpose.

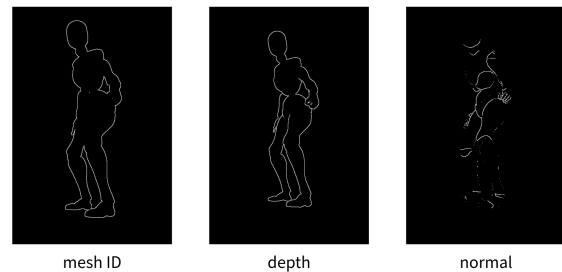


Figure 5: Contour detection

2.2.4 Perlin noise

To distort the contour, we used perlin noise, as it is a simple and efficient method to add "continuous randomness". To have a smooth animation, we encoded time as

a third variable for the perlin noise generation. To have the distortion, we added our 2D noise to the texCoord in the final render pass' fragment shader and used this new texCoord to compute the contour. You can see the result in figure 6. Several parameters (uniforms in the shader) can be changed to have higher/lower peaks or fatter/slower movements.

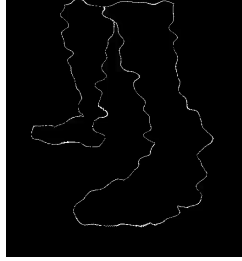


Figure 6: sketchy effect with perlin noise

As we already implemented perlin noise, it was easy to implement additional effects, more precisely a procedural moving color background and a distortion of the scene with the same trick as for the contour distortion, giving the impression to be under water, as you can see in figure 7.

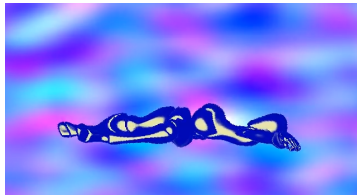


Figure 7: Additional effects with perlin noise

2.3 Fragment shaders

We implemented the two primary two-dimensional primitives for toon shading within the fragment shader: *halftoning* (dot patterns) and *crosshatching* (line patterns). These procedural textures are driven by Signed Distance Functions (SDFs) combined with the built-in GLSL `fract()` function, which enables efficient modulo operations on float variables.

To achieve a visually compelling and stylized aesthetic, we leverage the three standard components of the lighting model. Inspired by traditional animation and cinematography techniques, the thickness of the primitives scales dynamically with the intensity of each light component:

- **Ambient light:** Associated with *crosshatching* to define shadow regions and core dark zones.
- **Diffuse light:** Left clear of any primitives to maintain surface readability and brightness.
- **Specular light:** Associated with *halftoning* to stylize highlights and bright reflections.

Furthermore, to ensure seamless and aesthetically pleasing transitions where different lighting components overlap or dominate, we apply a sinusoidal *smoothstep* interpolation. Here is the result :

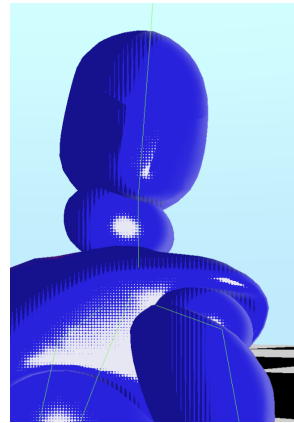


Figure 8: Enter Caption

To confer a painterly, artistic style to the final image, we implemented an anisotropic Kuwahara filter as a post-processing pass. The computation pipeline is structured into three distinct stages:

1. **Structure Tensor Extraction:** The pipeline reads from the final image buffer to compute a local structure

tensor for each pixel. This tensor is represented as a 2×2 matrix derived from local intensity gradients.

2. **Tensor Smoothing:** Because raw gradient data is inherently noisy, a standard Gaussian filter is applied to the structure tensor buffer to smooth the tensor field and ensure spatial coherence.
3. **Kuwahara Filter Application:** Reading from both the smoothed tensor buffer and the original image buffer, we calculate the eigenvalues of the structure tensor to determine local anisotropy. These metrics are used to construct a transformation matrix that warps the filter kernel, extending it along the direction of structural edges. The local neighborhood is then subdivided into 8 angular sectors. For each sector, we evaluate a Gaussian-weighted mean and variance. The final pixel color is determined by the sector exhibiting the minimum variance, effectively preserving sharp boundaries while smoothing out minor details.

The final painterly effect achieved through this post-processing pipeline is shown under :

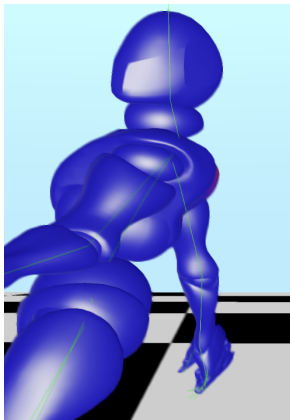


Figure 9: Enter Caption

The result is more deceptive than the

toon shading. We implemented it at the end of the project and we couldn't test it much and finetune it to have a great result.

3 Conclusion

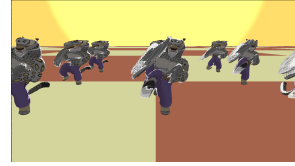


Figure 10: Final scene

We implemented various methods to have different stylizations. We reunited our works in a single animation that you can see figure 10.

References

- [1] Jean Basset, Pierre Bénard, and Pascal Barla. "Smear: Stylized motion exaggeration with art-direction". In: *ACM SIGGRAPH 2024 Conference Papers*. 2024, pp. 1–11.
- [2] Curtis. "Loose and Sketchy Animation". In: 1998.
- [3] Amine Farhat et al. "Motion ribbons: Parametrized surfaces for depicting motion effects". In: *Computers & Graphics* 129 (2025), p. 104227.
- [4] hertzmann. "ntroduction to 3D Non-Photorealistic Rendering: Silhouettes and Outlines". In: 1990.
- [5] Raskar and Cohen. "Image Precision Silhouette Edges". In: 1999.
- [6] T. SAITO and T. TAKAHASHI. "Comprehensive rendering of 3-d shapes. Computer Graphics, SIGGRAPH". In: 1990.